

**Data Driven Evaluation
and Prediction of Striker
Performances in the
Premier League**

by

Yaman Serhatlıoğlu

Table of Contents

I.	Introduction.....	3
II.	Data Collection.....	4
III.	Data Cleaning.....	5
IV.	Basic Statistical Analysis.....	8
V.	Normalized and Efficiency Metrics.....	9
VI.	Trend Analysis.....	12
VII.	Predictive Modeling.....	15
VIII.	Custom Performance Index.....	18
IX.	Results.....	21
X.	Conclusion.....	26
XI.	References.....	27

I. Introduction

Overview of the Project

This project explores the intersection of sports analytics and machine learning through the evaluation of striker performance in the English Premier League (EPL). Focusing on the top 10 teams over three completed consecutive seasons (2022-23, 2023-24 and 2024-25), the project utilized Python to process, normalize and analyze sports data. I aimed to move beyond surface level goal and assist statistics to see deeper into performance trends by inspecting detailed, specific and relevant data. The project also includes a development of predictive models and a custom evaluation metric to measure attacking efficiency with maximum accuracy.

Why Football Performance Analysis is Suitable for Data Analysis and Machine Learning

Modern football generates massive, complicated data sets including everything from basic match outcomes to incredibly specific event by event data such as expected goals, shot location and minute by minute action mapping. As player performance is influenced by quantifiable variables, it is an ideal candidate for statistical analysis and machine learning. Regression models and analysis can identify hidden patterns within data and predict future outputs based on the true performance a player brings to the pitch.

Justification for the Focus of the Project

I selected Strikers of the top 10 EPL teams as the point of focus because they are consistently exposed to high pressure environments, elite tactical setups and in general more attacking opportunities. Only looking at the top 10 of the table drew a base line of quality of teams and allowed for a more focused comparison of individual performance.

Explanation of the Approach

As both team standings and squad rosters are dynamic through a season, a strict team based approach would be inconsistent for this project. Instead of this, I decided on a role based approach and only included strikers who played for whichever teams finished in the top 10 during a given season. This data set allowed for fair comparisons among different seasons and minute distributions by converting cumulative statistics into normalized efficiency ratings.

II. Data Collection

Identification of Data Sources

The project relied on two primary data sources:

- [Premier League Official Statistics](#) : Used for verifying final league standings, team goal differences and basic player statistics
- [Kaggle](#) : Used to access public data sets including advanced metrics (expected goals (xG), shot creation actions, per 90 statistics and detailed playing time) from [FBref Database](#)

Selection of Seasons and Top 10 Teams

The analysis focused on three completed seasons and the top 10 teams independently based on the final league table of each respective year:

- 2022-23: Manchester City, Arsenal, Manchester United, Newcastle United, Liverpool, Brighton & Hove Albion, Aston Villa, Tottenham, Brentford, Fulham.
- 2023-24: Manchester City, Arsenal, Liverpool, Aston Villa, Tottenham, Chelsea, Newcastle United, Manchester United, West Ham United, Crystal Palace.

- 2024-25: Liverpool, Arsenal, Manchester City, Chelsea, Newcastle United, Aston Villa, Nottingham Forest, Brighton & Hove Albion, Bournemouth, Brentford.

Implementation - [The Entire Code Can be Accessed Here](#)

I found data from public data bases on Kaggle based on FBref in order to conduct an analysis across three different seasons. Because data on Kaggle was stored in varying formats depending on who scraped/compiled it, I had to load three structurally different CSV files into a Python environment in Google Colab using the pandas library.

```
# LOAD DATA SETS

try:
    # 2022-23 Data (Match by match logs)
    df_raw_22 = pd.read_csv('FPL_logs.csv', low_memory = False)

    # 2023-24 Data (European Season Summaries)
    df_raw_23 = pd.read_csv('cleaned_2023-24.csv')

    # 2024-25 Data (Premier League Season Summary)
    df_raw_24 = pd.read_csv('fbref_PL_2024-25.csv')

    print("All three data sets loaded successfully.")

except FileNotFoundError as e:
    print(f"Error loading files: {e}. Make sure all 3 CSVs are uploaded.")
```

I used “pd.read_csv()” to import the data sets. For the 22-23 data, which contained dense match by match logs rather than seasonal summaries, I set the “low_memory = False” parameter to make sure that pandas could allocate sufficient memory.

III. Data Cleaning

Applying a universal cleaning script was impossible because the three data sets had different initial structures (the first one included match logs, the second one had

different European leagues' summaries and the third one was about specific Premier League summaries). So instead of that, I cleaned data in three different ways to have a standardized format. But across all data, I had two common filters of the position and the minutes threshold. I filtered the data so that the player's listed position must contain "FW" (forward) and the player must have played a minimum of 900 minutes (equivalent to ten full matches) in the Premier League.

Cleaning of the 2022-23 Data Set

```
# 1. Clean 2022-23 Data
# Filter for 22-23 season
df_22 = df_raw_22[df_raw_22['Season'] == '2022-23'].copy()

# Aggregate match by match data into season totals
df_22_agg = df_22.groupby(['Name', 'Team']).agg({'Min': 'sum', 'Gls': 'sum', 'Ast': 'sum', 'Sh': 'sum', 'xG': 'sum', 'Pos': 'first'}).reset_index()

# Standardize columns
df_22_agg.rename(columns={'Name': 'Player', 'Team': 'Squad', 'Sh': 'Shots'}, inplace=True)
df_22_agg['Season'] = '2022-2023'

top_10_22 = ['Manchester City', 'Arsenal', 'Manchester Utd', 'Newcastle Utd', 'Liverpool', 'Brighton', 'Aston Villa', 'Tottenham', 'Brentford', 'Fulham']

# Filter for strikers in top 10 teams
df_clean_22 = df_22_agg[(df_22_agg['Squad'].isin(top_10_22)) & (df_22_agg['Pos'].str.contains('FW', case=False, na=False)) & (df_22_agg['Min'] >= 900)].copy()
```

The "FPL_logs.csv" data set, which I have used to get the 22-23 data, contained a row for every single match a player participated in. To convert this into a usable season summary, I filtered for the "2022-23" season and used the ".groupby()" function. By grouping "Name" and "Team", I could aggregate the sum of their "Min" (minutes played), "Gls" (goals), "Ast" (assists), "Sh" (shots) and "xG" (expected goals).

Cleaning of the 2023-24 Data Set

```
# 2. Clean 2023-24 Data
# Filter for Premier League
df_23 = df_raw_23[df_raw_23['comp'] == 'Premier League'].copy()

# Standardize columns
df_23.rename(columns={'player': 'Player', 'squad': 'Squad', 'Avg Mins per Match': 'Min', 'Goals': 'Gls', 'Assists': 'Ast', 'Expected Goals': 'xG', 'Total Shots': 'Shots', 'pos': 'Pos'}, inplace=True)
df_23['Season'] = '2023-2024'

top_10_23 = ['Manchester City', 'Arsenal', 'Liverpool', 'Aston Villa', 'Tottenham', 'Chelsea', 'Newcastle Utd', 'Manchester Utd', 'West Ham', 'Crystal Palace']

# Filter for strikers in top 10 teams
df_clean_23 = df_23[(df_23['Squad'].isin(top_10_23)) & (df_23['Pos'].str.contains('FW', case=False, na=False)) & (df_23['Min'] >= 900)].copy()
```

The “cleaned_2023-24.csv” data set, which I have used to get the 23-24 data, contained multiple European leagues. So I used boolean indexing (`df['comp'] == 'Premier League'`) to isolate the English Premier League. I then renamed the poorly labeled columns and specifically corrected the ‘Avg Mins per Match’ column to ‘Min’ as it actually represented total minutes played.

Cleaning of the 2024-25 Data Set

```
# 3. Clean 2024-25 Data
# Standardize columns
df_raw_24.columns = df_raw_24.columns.str.strip()
df_raw_24['Player'] = df_raw_24['Player'].str.strip()
df_raw_24['Squad'] = df_raw_24['Squad'].str.strip()
df_raw_24['Season'] = '2024-2025'

# This data set does not have "Shots", so I fill it with nan.
df_raw_24['Shots'] = np.nan

top_10_24 = ['Liverpool', 'Arsenal', 'Manchester City', 'Chelsea', 'Newcastle Utd', 'Aston Villa', 'Nott'ham Forest', 'Brighton', 'Bournemouth', 'Brentford']

# Filter for strikers in top 10 teams
df_clean_24 = df_raw_24[(df_raw_24['Squad'].isin(top_10_24)) & (df_raw_24['Pos'].str.contains('FW', case=False, na=False)) & (df_raw_24['Min'] >= 900)].copy()
```

The “fbref_PL_2024-25.csv” data set, which I have used to get the 24-25 data, was well formatted but lacked a ‘Shots’ column. To maintain uniformity, I inserted a blank ‘Shots’ column using “`np.nan`” (Not a Number) from the numpy library.

Creating a Clean Data Set Suitable for Analysis

```
# Merging all seasons into one master data set
core_columns = ['Season', 'Player', 'Squad', 'Pos', 'Min', 'Gls', 'Ast', 'Shots', 'xG']
master_df = pd.concat([df_clean_22[core_columns], df_clean_23[core_columns], df_clean_24[core_columns]], ignore_index=True)
```

After cleaning and filtering for the respective top 10 teams of each season, I used “pd.concat()” to stack the three dataframes vertically into a single “master_df”.

IV. Basic Statistical Analysis

```
# BASIC STATISTICAL ANALYSIS

print("=== Top 10 Raw Goalscoring Seasons (2022-2025) ===")

# Sort the master dataframe by raw goals to see who dominated the league
top_scorers = master_df.sort_values(by='Gls', ascending=False).head(10)
display(top_scorers[['Season', 'Player', 'Squad', 'Min', 'Gls', 'Ast']])

print("\n=== Summary Statistics by Season ===")

# Show the average goals and minutes played by Top 10 team strikers each year
season_summary = master_df.groupby('Season').agg({'Player': 'count', 'Min': 'mean', 'Gls': 'mean', 'Ast': 'mean'}).rename(columns={'Player': 'Number of Qual. Strikers', 'Min': 'Avg Mins', 'Gls': 'Avg Goals'})

display(round(season_summary, 1))
```

Before developing complex metrics, I conducted a surface level statistical analysis of the “master_df” to have a baseline. This analysis involves identifying the highest goalscorers across the three year window by using the “.sort_values()” method on the raw “Gls” column. Furthermore, I grouped the master data set by season and calculated the .mean() for minutes and goals. This provides context of how the average performance of a top 10 striker fluctuates over years. The results can be seen below:

=== Top 10 Raw Goalscoring Seasons (2022-2025) ===						
	Season	Player	Squad	Min	Gl's	Ast
16	2022-2023	Erling Haaland	Manchester City	2769.0	36.0	8.0
18	2022-2023	Harry Kane	Tottenham	3405.0	30.0	3.0
125	2024-2025	Mohamed Salah	Liverpool	3371.0	29.0	18.0
55	2023-2024	Erling Haaland	Manchester City	2552.0	27.0	5.0
101	2024-2025	Alexander Isak	Newcastle Utd	2756.0	23.0	6.0
98	2024-2025	Erling Haaland	Manchester City	2736.0	22.0	3.0
73	2023-2024	Cole Palmer	Chelsea	2607.0	22.0	11.0
58	2023-2024	Alexander Isak	Newcastle Utd	2255.0	21.0	2.0
20	2022-2023	Ivan Toney	Brentford	2951.0	20.0	4.0
110	2024-2025	Bryan Mbeumo	Brentford	3414.0	20.0	7.0
=== Summary Statistics by Season ===						
	Number of Qual. Strikers		Avg Mins	Avg Goals	Ast	
Season						
2022-2023	41		1947.0	9.3	4.4	
2023-2024	43		2052.3	9.7	5.0	
2024-2025	54		2133.5	7.9	4.7	

V. Normalized and Efficiency Metrics

Relying on cumulative statistics like raw goals introduces an inherent bias favoring players with more playing time. So to make objective comparisons between strikers who played 3000 minutes and those who played 1200, I applied mathematical normalization.

Per 90 Normalization

```
# NORMALIZED AND EFFICIENCY METRICS

# 1. Calculate Per 90 Metrics for fair comparison
master_df['Gls_per_90'] = round((master_df['Gls'] / master_df['Min']) * 90, 2)
master_df['Ast_per_90'] = round((master_df['Ast'] / master_df['Min']) * 90, 2)
master_df['xG_per_90'] = round((master_df['xG'] / master_df['Min']) * 90, 2)
```

I calculated per 90 metrics by dividing the cumulative statistic by the player's total minutes and multiplying the result by 90 (the length of a standard match). This was achieved efficiently in pandas using vectorized operations rather than for loops.

Efficiency Metrics

```
# 2. Calculate Efficiency (Finishing Efficiency = Goals / Expected Goals (xG). > 1.0 means ruthless finishing.)
master_df['Finishing_Efficiency'] = round(master_df['Gls'] / (master_df['xG'] + 1e-9), 2)

# 3. Calculate Conversion Rate (Goals / Shots (Not applicable for the 24-25 season because of lack of data))
master_df['Conversion_Rate_%'] = round((master_df['Gls'] / (master_df['Shots'] + 1e-9)) * 100, 2)
```

To measure how clinical a striker is independent of their team's efficiency at chance creation, I thought of two efficiency metrics:

1. Conversion Rate (Goals / Shots): Measures the percentage of shots that result in a goal. (Unavailable for the 24-25 data set due to the missing 'Shots' column)
2. Finishing Efficiency (Goals / xG): A metric comparing actual goals to expected goals. A value greater than 1.0 indicates a player is outperforming statistical expectations (good finishing), while a value below 1.0 indicates poor finishing.

To prevent division by zero runtime errors (in the rare event a player had 0 xG or 0 shots), I added 1e-9, a value small enough to not affect the results but a value greater than zero nonetheless to prevent the error, to the denominators during calculation.

```
# 4. View the Most Clinical Finishers Across the 3 Years
# Filter for players who scored at least 5 goals to avoid anomalies
clinical_finishers = master_df[master_df['Gls'] >= 5].sort_values(by='Finishing_Efficiency', ascending=False)

view_cols = ['Season', 'Player', 'Squad', 'Gls', 'xG', 'Gls_per_90', 'Finishing_Efficiency']

print("=== Top 10 Most Clinical Strikers (Finishing Efficiency) ===")
display(clinical_finishers[view_cols].head(10))

print("\n=== Worst 5 Underperformers (Scored less than their xG) ===")
display(clinical_finishers[view_cols].tail(5))
```

Then, I created a subset called “clinical_finishers”. I printed the 10 best finishers in the Premier League using an assortment of finishing efficiency, as the conversion rate was unfortunately not available for all seasons. The worst 5 finishers were also displayed. The results could be seen in the table below:

=== Top 10 Most Clinical Strikers (Finishing Efficiency) ===

	Season	Player	Squad	Gls	xG	Gls_per_90	Finishing_Efficiency
37	2022-2023	Roberto Firmino	Liverpool	11.0	5.4	0.82	2.04
100	2024-2025	Callum Hudson-Odoi	Nott'ham Forest	5.0	2.5	0.21	2.00
65	2023-2024	Noni Madueke	Chelsea	5.0	2.5	0.43	2.00
62	2023-2024	Diogo Jota	Liverpool	10.0	5.3	0.79	1.89
34	2022-2023	Phil Foden	Manchester City	11.0	5.9	0.54	1.86
50	2023-2024	Phil Foden	Manchester City	19.0	10.3	0.60	1.84
72	2023-2024	Michael Olise	Crystal Palace	10.0	5.5	0.71	1.82
64	2023-2024	Dejan Kulusevski	Tottenham	8.0	4.7	0.26	1.70
110	2024-2025	Bryan Mbeumo	Brentford	20.0	12.3	0.53	1.63
63	2023-2024	Mohammed Kudus	West Ham	8.0	5.1	0.29	1.57

=== Worst 5 Underperformers (Scored less than their xG) ===

	Season	Player	Squad	Gls	xG	Gls_per_90	Finishing_Efficiency
107	2024-2025	Noni Madueke	Chelsea	7.0	9.6	0.31	0.73
71	2023-2024	Darwin Núñez	Liverpool	11.0	16.3	0.48	0.67
47	2023-2024	Luis Díaz	Liverpool	8.0	11.9	0.27	0.67
11	2022-2023	Danny Welbeck	Brighton	6.0	9.4	0.29	0.64
61	2023-2024	Brennan Johnson	Tottenham	5.0	10.3	0.22	0.49

VI. Trend Analysis - *(Findings from here on out has been documented in the results section)*

While normalized tables are good for rankings players, they do not actually reveal how performances change over time. So I transitioned from static data frames to dynamic time-series visualizations to see improvements and declines across the three seasons.

```
# TREND ANALYSIS

# Import visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Set a visual theme for the charts for clearer visuals
sns.set_theme(style="whitegrid", palette="muted")
print("Visualization libraries successfully imported.")
```

For this implementation, I used matplotlib with seaborn which is a statistical data visualization library that provides a readable and professional aesthetic.

League-Wide Macro Trends

Before zooming in on individual players, it is important to see the macro trends of the Premier League. I aimed to determine if the league's top attackers are collectively becoming more clinical or if simply only the expected goals are increasing.

```

# League-Wide Macro Striker Trends

# Calculate average goals per 90 and xG per 90 for each season
season_trends = master_df.groupby('Season')[['Gls_per_90', 'xG_per_90']].mean().reset_index()

# Set up matplotlib
fig, ax = plt.subplots(figsize=(10, 6))

# Plot both metrics as lines on the same graph
sns.lineplot(data=season_trends, x='Season', y='Gls_per_90', marker='o', linewidth=3, label='Actual Goals per 90', ax=ax)
sns.lineplot(data=season_trends, x='Season', y='xG_per_90', marker='s', linewidth=3, linestyle='--', label='Expected Goals (xG) per 90', ax=ax)

# Format the graph to make it readable
ax.set_title('Macro Trend: Average Striker Efficiency for Top 10 Teams of Premier League Through Seasons', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Season', fontsize=12, labelpad=10)
ax.set_ylabel('Metrics Per 90 Minutes', fontsize=12, labelpad=10)
ax.set_ylim(0, 1.0)
plt.legend(title='Metric', fontsize=11)

plt.tight_layout()

plt.show()

```

To achieve this, I aggregated the “master_df” using the “.groupby()” function to calculate the mean “Gls_per_90” (actual goals) and “xG_per_90” (expected goals) for all top 10 strikers within each season. Then, I plotted these two metrics on the same axis for direct comparison. The graph demonstrates whether the collective finishing ability of the strikers is over or underperforming the baseline chance quality.

Player Specific Micro Trends

```
# Individual Micro Striker Trends

# Identify a few players who have data across multiple seasons
# Could be changed to look into different players
target_players = ['Erling Haaland', 'Ollie Watkins', 'Bukayo Saka', 'Mohamed Salah']

# Filter the master data set for only these players
player_trends = master_df[master_df['Player'].isin(target_players)].copy()

# Set up matplotlib
fig, ax = plt.subplots(figsize=(12, 7))

# Plot a line graph that also colors the lines by player
sns.lineplot(data=player_trends, x='Season', y='Finishing_Efficiency', hue='Player', marker='o', markersize=10, linewidth=3, ax=ax)

# Add a line at 1.0 to reference baseline
ax.axhline(y=1.0, color='red', linestyle='--', alpha=0.5, label='Baseline (Goals = xG)')

# Format the graph to make it readable
ax.set_title('Player Performance Trends: Finishing Efficiency Over Time', fontsize=16, fontweight='bold', pad=15)
ax.set_xlabel('Season', fontsize=12, labelpad=10)
ax.set_ylabel('Finishing Efficiency (Goals / xG)', fontsize=12, labelpad=10)

# Move the legend outside the graph so it does not overlap data lines
plt.legend(title='Player', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()

plt.show()
```

In order to look into player specific performance throughout seasons, I had to create a subset of strikers who consistently met the minimum 900 minutes played requirement in top 10 teams across all three seasons (e.g. Erling Haaland, Ollie Watkins, Bukayo Saka, Mohamed Salah). While graphing, instead of plotting raw goals, I plotted strikers' finishing efficiency (goals / xG) to have a fair comparison between players who played more compared to those that have had a less time on pitch. An important component of this visualization is the baseline reference line at $y=1.0$ using “`ax.axhline()`”. This supports the viewer's understanding of the data as any data point above the red dashed line represents good finishing (scoring more than expected), while anything below it represents underperformance. The graph has multiple lines to represent different players, which helps me determine a specific player's performance across different seasons.

VII. Predictive Modeling

After setting up the descriptive trends of Premier League strikers, it is now time for transitioning into predictive analytics. My objective in this section was to determine if historical performance metrics can accurately predict a striker's future performance. To achieve this, I used a Multiple Linear Regression model built with the scikit-learn library.

Temporal Train-Test Split

In standard machine learning practice, data sets are often split randomly into training and testing subsets. However, because sports data is chronological, a random split risks the usage of future events to predict the past, which is something I did not want. So I implemented a Temporal Split instead.

```
# MACHINE LEARNING SETUP AND DATA SPLITTING

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Define features (independent variables) and target(dependent variable)
features = ['xG_per_90', 'Ast_per_90', 'Min']
target = 'Gls_per_90'

# Temporal data split to train on the past and test on the present data
train_data = master_df[master_df['Season'].isin(['2022-2023', '2023-2024'])].copy()
test_data = master_df[master_df['Season'] == '2024-2025'].copy()

# Seperate the features and the target for both sets
X_train = train_data[features]
y_train = train_data[target]
X_test = test_data[features]
y_test = test_data[target]

print(f"Training model on {len(X_train)} historical striker seasons...")
print(f"Testing model on {len(X_test)} current striker seasons...")
```

The methodology I used was like this; I trained a model entirely on historical data (the 22-23 and 23-24 season) and tested its prediction accuracy on the “unseen” rather present data (the 24-25 season).

Model Training and Evaluation

```
# Model Training and Evaluation

# Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the unseen 24-25 data
y_pred = model.predict(X_test)

# Evaluate model accuracy
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("=== Predictive Model Evaluation ===")
print(f"R-Squared (R²): {r2:.3f}")
print(f"Mean Absolute Error (MAE): {mae:.3f} goals per 90")
print(f"Root Mean Squared Error (RMSE): {np.sqrt(mse):.3f} goals per 90\n")

# Display the weight the model assigned to each feature
print("=== Feature Importance ===")
for feature, coef in zip(features, model.coef_):
    print(f"{feature}: {coef:.4f}")
```

I initialized a Multiple Linear Regression model and fit it to my training data. The model analyzes the relationship between the chosen features (expected goals per 90, assists per 90 and total minutes) to predict the target variable (actual goals per 90). After training, I prompted the model to predict the 2024-25 goal scoring rates based solely on the chosen metrics of that season. I evaluated its mathematical accuracy using three primary regression metrics:

- R-Squared (R^2): Measures the proportion of variance in goals scored that is predictable from my selected features.
- Mean Absolute Error (MAE): Represents the average magnitude of the prediction errors and gives me a solid “goals per 90” margin of error.
- Feature Importance (Coefficients): Identifies the specific mathematical weight the AI assigned to each variable (e.g. proving that xG is heavily weighted in predicting actual goals).

Predictions vs. Reality (Residual Analysis)

```
# Prediction Results Table

test_data['Predicted_Gls_per_90'] = np.round(y_pred, 2)

test_data['Prediction_Error'] = test_data['Gls_per_90'] - test_data['Predicted_Gls_per_90']

results_view = test_data[['Player', 'Squad', 'xG_per_90', 'Predicted_Gls_per_90', 'Gls_per_90', 'Prediction_Error']]

results_view = results_view.sort_values(by='Gls_per_90', ascending=False)

print("=== 2024-2025 Striker Predictions vs. Reality ===")
display(results_view)
```

Statistical metrics provide a mathematical summary of the model but they are not enough on their own to explain how accurate the model actually was. So I saved the model's predictions as a new column “Predicted_Gls_per_90” in my test data set and calculated the “Prediction_Error” (actual goals - predicted goals). This final “Prediction_Error” output actually could be seen as an independent performance indicator as well. For example, if a player has a highly positive prediction error, that means that they are converting low probability chances into goals. Conversely, a negative error shows players who get into high xG positions but consistently fail to finish the chance.

VIII. Custom Performance Index

Throughout this project, I have analyzed various independent metrics from raw goals to advanced finishing efficiencies. However, evaluating a striker holistically requires combining these different metrics into a single comprehensive rating. Because these metrics exist on different mathematical scales (e.g. goals per 90 averages around 0.50 while finishing efficiency often exceeds 1.00), I could not simply add them up as that would create a mathematically flawed metric that is weighted towards larger numbers. To solve this, I designed a Custom Performance Index (CPI). This is a weighted evaluation system scaled out of 100.

Mathematical Normalization

I restricted my final evaluation to the most recent 2024-25 season to rank active strikers. In order to get all statistical categories to weigh equally before applying a custom formula, I used the “MinMaxScaler” from scikit-learn.

```
from sklearn.preprocessing import MinMaxScaler

# Filter for the most recent season (24-25) to evaluate current strikers
current_strikers = master_df[master_df['Season'] == '2024-2025'].copy()

# Select the metrics that will make up the index
index_metrics = ['Gls_per_90', 'xG_per_90', 'Finishing_Efficiency', 'Ast_per_90']

# Initialize the Scaler (converts all stats to a 0-100 scale based on the highest/lowest in the league)
scaler = MinMaxScaler(feature_range=(0, 100))

# Create normalized columns
for metric in index_metrics:
    current_strikers[f'{metric}_Score'] = scaler.fit_transform(current_strikers[[metric]])
```

This algorithm evaluates the maximum and minimum values of a specific metric across the entire league and proportionally scales every player’s output to a continuous range

between 0 and 100. For example, the player with the highest goals per 90 receives a score of 100 in that category while the player with the lowest receives a 0.

Custom Weighting and Calculation

```
# Define custom weights
weights = {'Gls_per_90_Score': 0.40, 'xG_per_90_Score': 0.25, 'Finishing_Efficiency_Score': 0.20, 'Ast_per_90_Score': 0.15}

# Calculate the final Custom Performance Index (CPI)
current_strikers['CPI'] = ((current_strikers['Gls_per_90_Score'] * weights['Gls_per_90_Score']) +
                           (current_strikers['xG_per_90_Score'] * weights['xG_per_90_Score']) +
                           (current_strikers['Finishing_Efficiency_Score'] * weights['Finishing_Efficiency_Score']) +
                           (current_strikers['Ast_per_90_Score'] * weights['Ast_per_90_Score']))

# Round the index for clean viewing
current_strikers['CPI'] = round(current_strikers['CPI'], 1)
```

After normalizing all metrics to a standard 0-100 scale, I defined a custom weighting system that reflects the modern demands of a striker. The CPI assigns proportional importance to the following four dimensions:

- Goals per 90 (40%): The primary objective of a striker is to score. This is heavily weighted for it to proportionally affect the overall rating of a player according to its significance.
- Expected goals (xG) per 90 (25%): This is a statistic that rewards good offensive movement and ability to consistently find high probability goal scoring positions.
- Finishing efficiency (20%): Measures execution. Rewards players who maximize their chances regardless of their team's overall offensive quality.
- Assists per 90 (15%): Measures the playmaking and holistic contribution of a player to the attacking play.

Traditional vs. Advanced Ranking

```
# Traditional vs. Custom Index Comparison

# Create a traditional rank (based on raw goals)
current_strikers['Rank_Traditional'] = current_strikers['Gls'].rank(ascending=False, method='min')

# Create the custom rank (based on new CPI)
current_strikers['Rank_CPI'] = current_strikers['CPI'].rank(ascending=False, method='min')

# Calculate difference in ranks (positive means the index favors the player more than traditional stats)
current_strikers['Rank_Difference'] = current_strikers['Rank_Traditional'] - current_strikers['Rank_CPI']

# Select columns to display in the final report
final_columns = ['Player', 'Squad', 'Gls', 'Rank_Traditional', 'CPI', 'Rank_CPI', 'Rank_Difference']

# Sort by the new custom index
cpi_rankings = current_strikers[final_columns].sort_values(by='CPI', ascending=False)

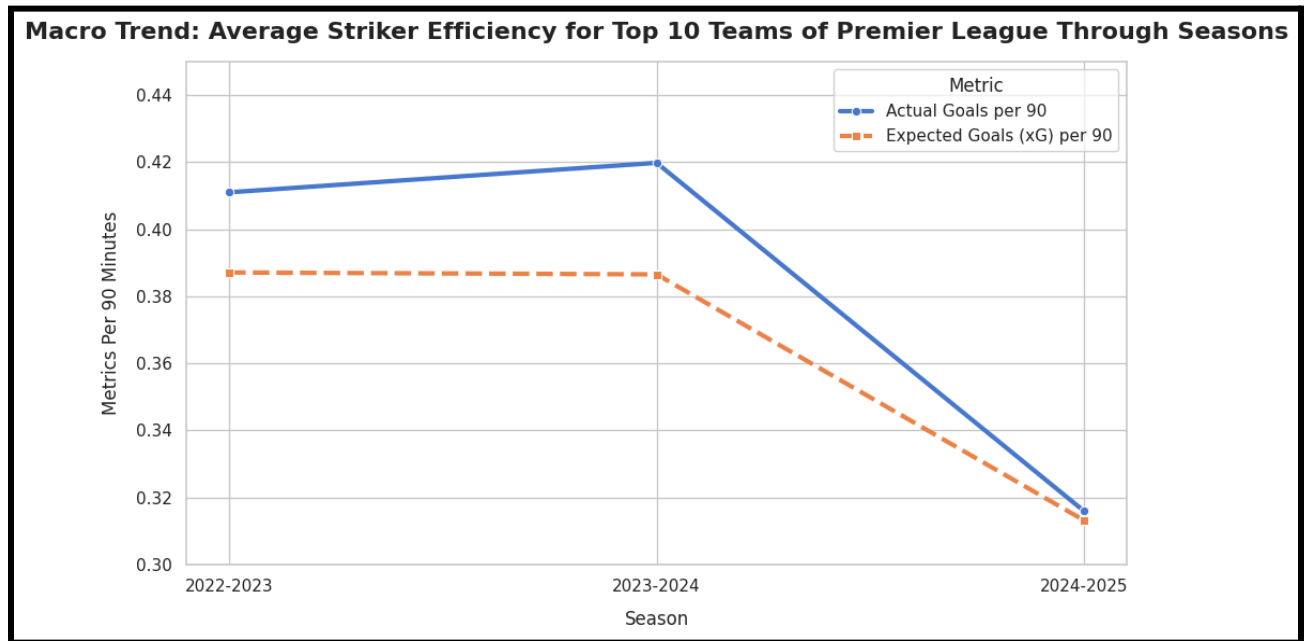
print("=== Top 15 Complete Strikers (Custom CPI Rankings) ===")
display(cpi_rankings.head(15))

print("\n=== Biggest 'Hidden Gems' (Players rated higher by CPI than by raw goals) ===")
hidden_gems = cpi_rankings.sort_values(by='Rank_Difference', ascending=False).head(5)
display(hidden_gems)
```

The true value of a custom data index like mine lies in its ability to uncover things that traditional metrics fail to measure. So I calculated a “Rank_Traditional” based only on amounts of raw goals and compared it against my “Rank_CPI”. I did this by subtracting the CPI rank from the traditional rank to create a “Rank_Difference” column. This difference being highly positive would identify a “hidden gem” player that may not necessarily lead the league in raw goals but has a high per 90 efficiency, chance creation and finishing skills. These “hidden gems” would ideally be revealed to be underrated complete strikers in the Premier League.

IX. Results

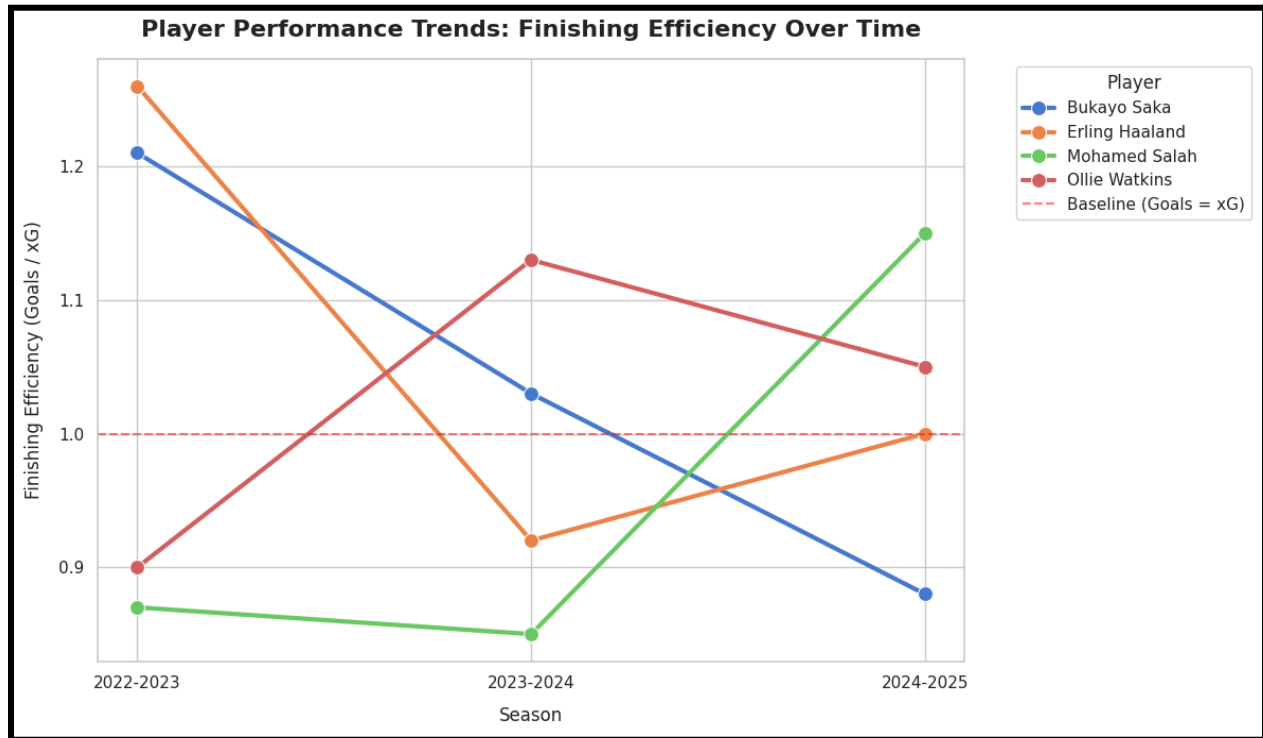
Macro Trends (The League)



As seen in the macro trends graph generated from the code, the Premier League's top 10 teams experienced a significant shift in offensive efficiency over the three seasons.

During the 22-23 and 23-24 seasons, strikers consistently overperformed their expected metrics by having an average of 0.41 to 0.42 actual goals per 90 minutes with respect to a baseline of 0.39 expected goals per 90. However, in the 2024-25 season, both chance creation (0.31 xG per 90) and the actual number of goals (0.32 goals per 90) dropped significantly. But despite this drop in volume, the strikers are still finishing within statistical expectations, which can be seen in the graph from the closeness of the two lines.

Micro Trends (The Players)



The individual player trajectory chart generated from the code shows different changes in finishing efficiency of different forwards. Bukayo Saka displays a distinct downward trend in finishing, dropping from a clinical 1.21 efficiency in 22-23 to an underperforming 0.88 by 24-25. Conversely, Ollie Watkins demonstrates a development. He transformed from a slightly wasteful finisher with an efficiency of 0.89 in his first year into an elite striker peaking at 1.13 before ending up with 1.05. On the other hand, Erling Haaland displays a dominating performance in the 22-23 season with an efficiency greater than 1.2, drops to a not-bad efficiency of around 0.93 in the 23-24 season and finally ends with 1.00 finishing efficiency in the 24-25 season. Mohamed Salah, who can also be seen in the graph, scored below his expected goals with efficiencies of 0.86 and 0.84 in the 22-23 and 23-24 seasons respectively. However, he exploded for a great 1.15 efficiency in the 24-25 season.

Predictive Model Performance

```
=== Predictive Model Evaluation ===  
R-Squared (R2): 0.739  
Mean Absolute Error (MAE): 0.079 goals per 90  
Root Mean Squared Error (RMSE): 0.097 goals per 90  
  
=== Feature Importance ===  
xG_per_90: 0.9093  
Ast_per_90: -0.0855  
Min: 0.0000
```

The Multiple Linear Regression model output confirms that the metrics I have used were robust predictors of actual performance. As seen in the model evaluation metrics, the algorithm achieved an R^2 score of 0.739, meaning it successfully explained nearly 75% of the variance in striker output in the 24-25 season only using xG, assists and minutes played from the 22-23 and 23-24 seasons. The Mean Absolute Error was unexpectedly low at only 0.079 goals per 90. The Root Mean Squared Error was also highly accurate with a value of 0.096 goals per 90. Looking at the feature importance data, the “xG_per_90” feature had a huge coefficient of 0.9093. This shows that the “xG_per_90” feature is by far the strongest predictor as essentially, for every 1.0 increase in a player’s expected goals per 90, their actual goal output rises by approximately 0.91. Interestingly, the “Ast_per_90” feature had a slightly negative coefficient of -0.0855, suggesting that players who had gotten more assist making chances had slightly less raw goal output. The total “Min” played had a coefficient of zero, which confirmed that my per 90 normalization successfully removed playing time bias.

=== 2024-2025 Striker Predictions vs. Reality ===						
	Player	Squad	xG_per_90	Predicted_Gls_per_90	Gls_per_90	Prediction_Error
125	Mohamed Salah	Liverpool	0.67	0.65	0.77	0.12
101	Alexander Isak	Newcastle Utd	0.66	0.67	0.75	0.08
98	Erling Haaland	Manchester City	0.72	0.73	0.72	-0.01
137	Chris Wood	Nott'ham Forest	0.41	0.45	0.61	0.16
136	Yoane Wissa	Brentford	0.57	0.59	0.59	0.00
133	Ollie Watkins	Aston Villa	0.53	0.54	0.55	0.01
108	Omar Marmoush	Manchester City	0.42	0.46	0.54	0.08
110	Bryan Mbeumo	Brentford	0.32	0.36	0.53	0.17
90	Luis Díaz	Liverpool	0.45	0.48	0.49	0.01
96	Cody Gakpo	Liverpool	0.33	0.37	0.47	0.10
86	Harvey Barnes	Newcastle Utd	0.37	0.40	0.46	0.06
103	João Pedro	Brighton	0.41	0.43	0.46	0.03
105	Diogo Jota	Liverpool	0.59	0.60	0.45	-0.15
99	Kai Havertz	Arsenal	0.46	0.49	0.43	-0.06
134	Danny Welbeck	Brighton	0.41	0.44	0.43	-0.01
128	Kevin Schade	Brentford	0.32	0.37	0.43	0.06
120	Cole Palmer	Chelsea	0.49	0.51	0.42	-0.09
102	Nicolas Jackson	Chelsea	0.50	0.52	0.41	-0.11
112	Mikel Merino	Arsenal	0.33	0.37	0.40	0.03
118	Darwin Núñez	Liverpool	0.46	0.49	0.40	-0.09
94	Evanilson	Bournemouth	0.49	0.52	0.39	-0.13
95	Phil Foden	Manchester City	0.26	0.31	0.35	0.04
114	Kaoru Mitoma	Brighton	0.32	0.36	0.35	-0.01
129	Antoine Semenyo	Bournemouth	0.28	0.33	0.31	-0.02
109	Gabriel Martinelli	Arsenal	0.29	0.33	0.31	-0.02
124	Bukayo Saka	Arsenal	0.35	0.36	0.31	-0.05
119	Dango Ouattara	Bournemouth	0.38	0.41	0.31	-0.10
115	Jacob Murphy	Newcastle Utd	0.22	0.24	0.31	0.07
107	Noni Madueke	Chelsea	0.42	0.45	0.31	-0.14
117	Christopher Nkunku	Chelsea	0.47	0.49	0.29	-0.20
113	Yankuba Minteh	Brighton	0.20	0.25	0.29	0.04
132	Leandro Trossard	Arsenal	0.25	0.29	0.28	-0.01

The prediction error table revealed the outliers contradicting the mathematical models. Bryan Mbeumo and Chris Wood were the league's biggest overperformers in the test data, scoring 0.17 and 0.16 goals per 90 more than the AI predicted, respectively. On the

opposite side, Savio and Nkunku heavily struggled as they scored 0.22 and 0.20 goals per 90 below the model's expectations, respectively.

The Custom Index (CPI) Findings

=== Top 15 Complete Strikers (Custom CPI Rankings) ===							
	Player	Squad	GLs	Rank_Traditional	CPI	Rank_CPI	Rank_Difference
125	Mohamed Salah	Liverpool	29.0	1.0	86.9	1.0	0.0
101	Alexander Isak	Newcastle Utd	23.0	2.0	77.2	2.0	0.0
98	Erling Haaland	Manchester City	22.0	3.0	73.9	3.0	0.0
133	Ollie Watkins	Aston Villa	16.0	7.0	63.4	4.0	3.0
136	Yoane Wissa	Brentford	19.0	6.0	62.3	5.0	1.0
137	Chris Wood	Nott'ham Forest	20.0	4.0	60.3	6.0	-2.0
105	Diogo Jota	Liverpool	6.0	29.0	56.6	7.0	22.0
110	Bryan Mbeumo	Brentford	20.0	4.0	56.5	8.0	-4.0
90	Luis Díaz	Liverpool	13.0	9.0	54.9	9.0	0.0
103	João Pedro	Brighton	10.0	12.0	54.7	10.0	2.0
108	Omar Marmoush	Manchester City	7.0	24.0	52.5	11.0	13.0
86	Harvey Barnes	Newcastle Utd	9.0	18.0	52.3	12.0	6.0
96	Cody Gakpo	Liverpool	10.0	12.0	52.1	13.0	-1.0
120	Cole Palmer	Chelsea	15.0	8.0	52.1	13.0	-5.0
102	Nicolas Jackson	Chelsea	10.0	12.0	50.6	15.0	-3.0
=== Biggest 'Hidden Gems' (Players rated higher by CPI than by raw goals) ===							
	Player	Squad	GLs	Rank_Traditional	CPI	Rank_CPI	Rank_Difference
105	Diogo Jota	Liverpool	6.0	29.0	56.6	7.0	22.0
117	Christopher Nkunku	Chelsea	3.0	41.0	41.6	24.0	17.0
118	Darwin Núñez	Liverpool	5.0	34.0	47.8	19.0	15.0
91	Jeremy Doku	Manchester City	3.0	41.0	40.1	27.0	14.0
108	Omar Marmoush	Manchester City	7.0	24.0	52.5	11.0	13.0

As seen in the final Custom Performance Index (CPI) rankings table, Mohamed Salah, Alexander Isak and Erling Haaland claimed the top three spots. This proves that the league's highest raw goal scorers were not just benefiting from sheer volume and that they genuinely had the highest efficiencies and abilities. However, the true value of the index can be seen in the "hidden gems" output. For example, Diogo Jota ranked a low

29th in traditional raw goals but the CPI elevated him 22 spots to become the 7th best overall forward in the league thanks to his successful per 90 metrics. Similarly, Omar Marmoush leaped 13 spots from his traditional rank, sitting at a new rank of 11th. The index proves that traditional raw goal statistics may sometimes mask effective hidden performances.

X. Conclusion

Strengths and Limitations

The primary strength of this approach lies in its rigorous normalization and feature engineering. By strictly forcing a 90 minute threshold and converting all statistics to per 90 rates, the analysis eliminated the bias that favors high minute players and the machine learning model worked successfully. However, real world football data presents inevitable limitations. The models cannot account for the quality of a player's teammates, tactical changes by managers, or injuries that may impact a player's physical form. The models also track all attack players in the Premier League, including both strikers and wingers, and that also produces errors or bias as players with different attacking positions tend to have different tasks and goals in the field, having different opportunities.

Future Extensions

If this project were to be expanded, having spatial data would be the next logical step. Feeding shot location coordinates into a predictive model could further improve the expected goals analysis and lower the RMSE even more. Additionally, grouping attack players into different archetypes like wingers, target men, inside forwards, etc. could allow for stronger and more accurate performance predictions.

XI. References

[The entire Python code regarding the implementation of the project can be accessed here](#). However, the code will not run without the necessary CSV files including the data sets, which could be found at: [CSA YHP Data Sets](#) .

The data sets for this project were taken from Kaggle for free. The following data sets, which all adapted data from the FBref football database, were used:

- [FPL-player-logs](#) by **Franek Sakowski** | for the 2022-23 season
- [FBref 2017-2024 for Europe's Top 5 leagues](#) by **Akshan Krithick** | for the 2023-24 season
- [FBREF Premier league 2024/25 player stats dataset](#) by **Siddhraj Thakor** | for the 2024-25 season

I have benefited vastly from [Keith Galli's Complete Python Pandas Data Science Tutorial! \(2025 Updated Edition\)](#), as well as [NeuralNine's Scikit-Learn Full Crash Course - Python Machine Learning](#), in order to be able to use pandas and scikit-learn libraries effectively in my project as I learned these concepts from zero while completing this project. For seaborn, I used [the tutorial tab in the official seaborn website](#) to have a grasp on the visualization.

[Google Gemini](#) assisted me in various different ways throughout the project as I used it to ask questions about the new libraries I have been working on and how I could implement some concepts in my own project. I also used Gemini when debugging during the times I got stuck and failed to find bugs. It helped me with the implementation of seaborn into my project as I asked for advice on making the visualization appear more professional and I particularly needed help with moving the

legend outside of the data visuals. In addition, I got help about the logic of creating a custom weighting system while constructing the Custom Performance Index.

Below, MLA citations of the sources used may be viewed.

Works Cited

Galli, Keith. "Complete Python Pandas Data Science Tutorial! (2025 Updated Edition)."

YouTube, 10 Mar. 2025, www.youtube.com/watch?v=1yir9_XFnLg. Accessed 20 Apr. 2026.

Gemini. Google, gemini.google.com. Accessed 20 Apr. 2026.

Krithick, Akshan. "FBref 2017-2024 for Europe's Top 5 leagues." Kaggle,

www.kaggle.com/datasets/akshankrithick/fbref-2017-2024-for-europes-top-5-leagues.
Accessed 20 Apr. 2026.

NeuralNine. "Scikit-Learn Full Crash Course - Python Machine Learning." YouTube, 3 Aug.

2025, www.youtube.com/watch?v=SIEaLBXr0rk. Accessed 20 Apr. 2026.

Sakowski, Franek. "FPL-player-logs." Kaggle,

www.kaggle.com/datasets/franeksakowski/fpl-player-logs. Accessed 20 Apr. 2026.

Thakor, Siddhraj. "FBREF Premier league 2024/25 player stats dataset." Kaggle,

www.kaggle.com/datasets/siddhrajthakor/fbref-premier-league-202425-player-stats-dataset.
et. Accessed 20 Apr. 2026.

"User Guide and Tutorial." Seaborn 0.13.2 Documentation, PyData,

seaborn.pydata.org/tutorial.html. Accessed 20 Apr. 2026.